

Kafka Topics Administration

Deep-Dive Training for Kafka Administrators

From logical data streams to production-safe topic operations.
Running example: orders.events

KAFKA ADMINISTRATORS

PLATFORM ENGINEERS

DEVOPS / SRE



What You Will Be Able to Do by the End

01

Explain core internals

Topics, partitions, offsets, replicas, leaders and ISR — with confidence

03

Operate safely

Create, inspect, modify, expand and delete topics using production-safe runbooks

02

Design for real workloads

Partition and replication decisions backed by measured inputs and formulas

04

Troubleshoot and govern

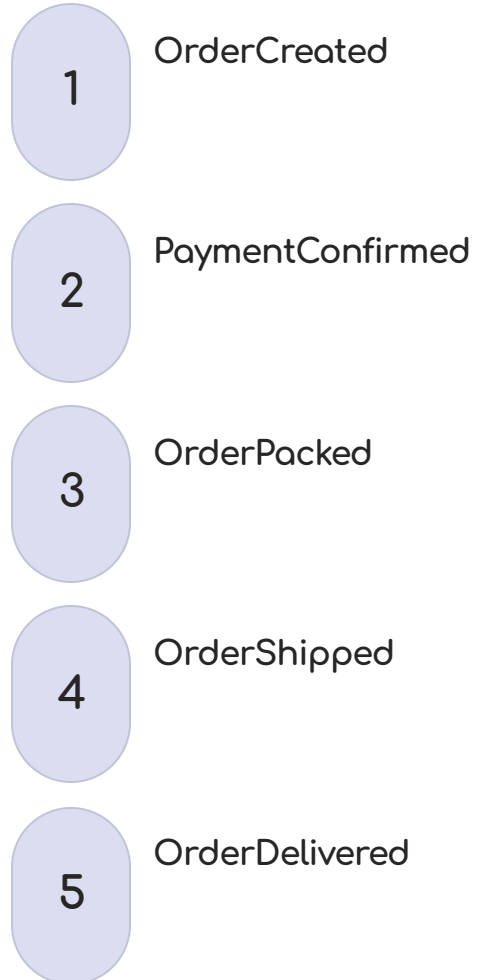
Diagnose under-replication, skew and outages; apply naming, security and lifecycle standards

Meet `orders.events`

Every concept in this training is anchored to a real platform you will design, build and operate today.

The Order Event Platform

An e-commerce platform publishes every stage of an order's life as a Kafka record on one topic: `orders.events`



What the Platform Requires from Administrators

Order Integrity

Events for each `orderId` must stay in order — always

Fault Tolerance

A single broker failure must not make the topic unavailable

Seven-Day Retention

Consumers must be able to replay a full week of events

Independent Consumers

Fraud, inventory, notifications and analytics all read independently

Growth-Ready

Design must absorb future traffic increases without emergency surgery

❏ Every design decision you make today must satisfy all five requirements simultaneously.

Topic → Partition → Offset

Build your mental model before touching a single command.

A Topic Is Not a Queue

Traditional Queue

- Message removed after consumption
- Consumer position managed by broker
- Replay is limited or special-case
- Multiple apps compete for messages

Kafka Topic

- Records retained by configurable policy
- Consumer position tracked as offsets
- Replay is a native, first-class feature
- Multiple groups read independently

Key insight: Kafka separates data retention from consumer progress. The topic does not care where consumers are.

A Topic Is Divided into Partitions



Each partition is a separate, ordered, append-only log. Partitions enable horizontal scalability, parallel reads/writes and distribution across brokers. The partition — not the topic — is Kafka's unit of ordering and parallelism.

Offsets: A Record's Exact Address

Unique within one partition

Offset 1057 on Partition 2 is different from offset 1057 on Partition 0

Monotonically increasing

Always starts at 0, always grows — never resets when old segments are deleted

Assigned after write

The broker assigns the offset when the record lands in the partition log

Example: `orders.events-2@1057` means topic `orders.events`, Partition 2, Offset 1057. Consumers commit this address to track where they stopped reading.

Ordering Stops at the Partition Boundary

What Kafka guarantees

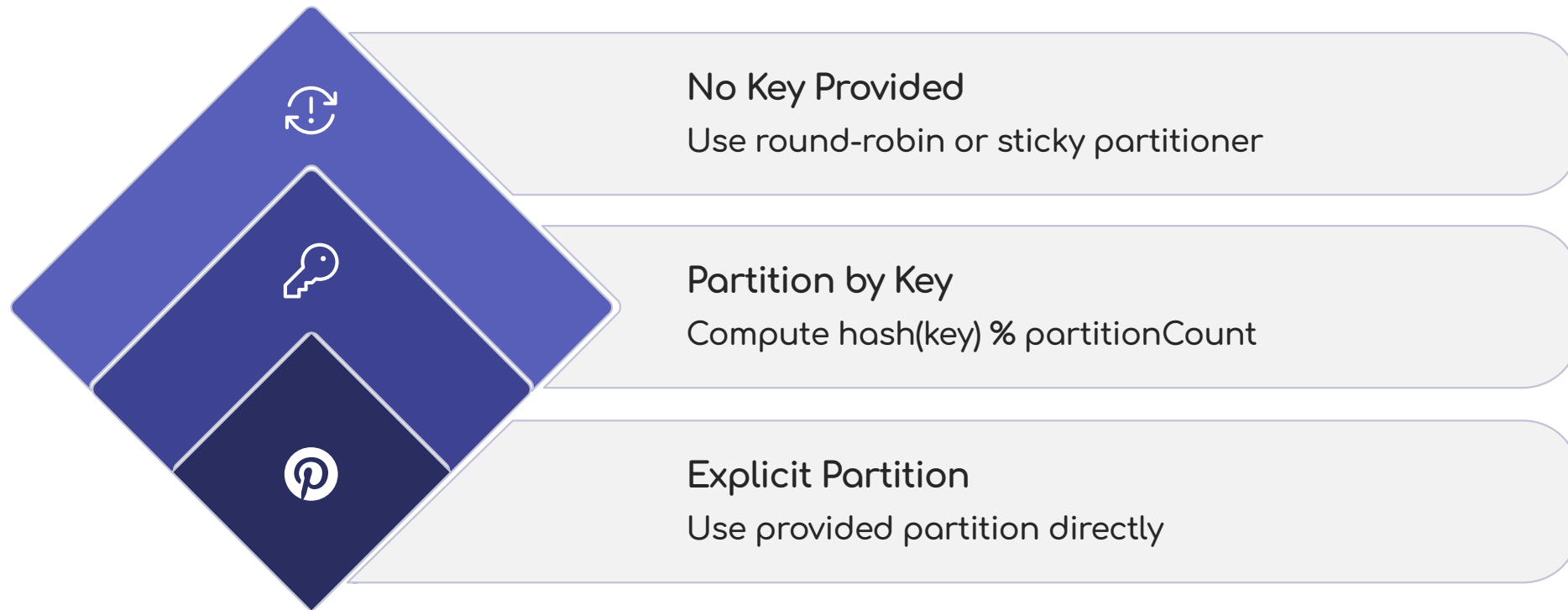
- Records within one partition are ordered
- Records with the same key go to the same partition
- Events for `ORD-9001` remain ordered — if keyed by `orderId`

What Kafka does not guarantee

- No automatic global order across all partitions
- Partition 0 and Partition 1 are independent logs

 If partition count changes, the key-to-partition mapping changes for future records. Historical and new records for one key may end up in different partitions.

How a Producer Picks a Partition



For `orders.events` the producer uses `orderId` as the key. The formula `partition = hash(orderId) % partitionCount` guarantees all events for one order land in the same partition — and in order. Changing `partitionCount` breaks this mapping for future records.

Checkpoint: Predict the Outcome

Scenario: orders.events has 6 partitions. Order ORD-5050 has events at offsets 12, 18 and 24 on Partition 3. You increase the topic to 9 partitions.

Question: Where will the *next* event for ORD-5050 land?

A

Still on Partition 3 — Kafka preserves key placement

B

On a new partition (7, 8 or 9) — new partitions get new data

C

Anywhere — hash result changes because partitionCount changed

Discuss with your neighbour. Answer is on the next card.

Answer: The Key Mapping Shifts

Correct answer: C

$\text{hash}(\text{ORD-5050}) \% 6 = 3$

$\text{hash}(\text{ORD-5050}) \% 9 = ?$ (likely a different partition)

Historical records remain on Partition 3. Future records land elsewhere.

Why administrators must care

- Stateful consumers may process the key out of order across the expansion boundary
- Coordinate every partition increase with application owners
- Build in enough headroom at creation time

Partition Count Is a Capacity Decision

There is no universal "correct number." Every choice must follow measured evidence.

Five Dimensions That Govern Partition Count



Start with Measured Inputs — Not Guesses

Input	orders.events Example
Peak records per second	60,000 rec/s
Average serialised record size	2 KB
Peak ingress (MB/s)	120 MB/s
Required consumer groups	4 groups
Slowest group capacity per instance	10 MB/s
Expected annual growth	40 %
Replication factor	3

 Always use peak traffic and serialised/compressed byte rates — not averages from a quiet period.

The Core Partition Formulas

Write-side requirement

$$P_{write} = \frac{\text{Peak ingress (MB/s)}}{\text{Safe write rate per partition (MB/s)}}$$

Read-side requirement (per group)

$$P_{read} = \frac{\text{Required group throughput (MB/s)}}{\text{Safe processing rate per consumer (MB/s)}}$$

Initial result

$$Partitions = \max(P_{write}, \text{highest } P_{read}, \text{required concurrency})$$

Then adjust for ordering, skew, growth, broker count and operational limits.

Worked Calculation: Write Side

Input

Peak ingress: 120 MB/s — Tested
safe write rate: 15 MB/s per
partition

Formula

$$P_{write} = \frac{120}{15} = 8 \text{ partitions}$$

What "safe" means

Meets produce latency SLO ·
Replication keeps up · CPU and
disk are stable · No growing
request queue · No ISR
instability under load



Do not use the highest rate seen in a short burst test. Measure the sustainable rate under realistic, sustained load.

Worked Calculation: Read Side

Fraud Detection Group

Must process 120 MB/s. One instance safely handles 10 MB/s.

$$P_{read} = \frac{120}{10} = 12 \text{ partitions}$$

Why 12 wins

Write side: 8 partitions needed.
Read side: 12 needed. We take the maximum → 12 partitions

The consumer parallelism rule

Within one consumer group, one partition is assigned to at most one active consumer. You cannot have more active workers than partitions.

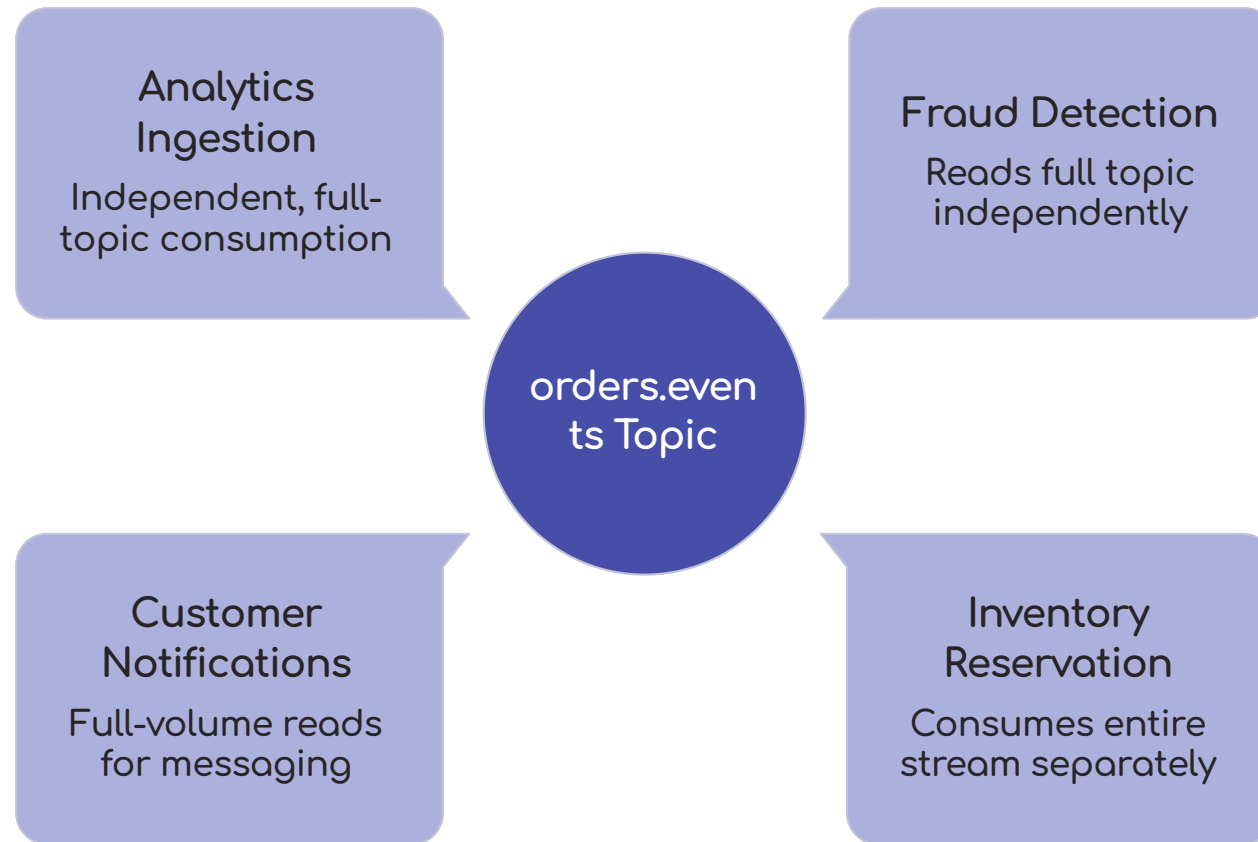
Consumer Parallelism: Idle Consumers Cost Money

With 12 partitions and one consumer group:

Consumers in group	Result
4	~3 partitions per consumer — under-utilised
12	1 partition per consumer — optimal
16	12 active, 4 idle — wasted resources

Multiple consumer groups do not share this limit. Each group independently reads all partitions. Having 4 groups each with 12 consumers needs 48 consumer processes total — but still only 12 partitions.

Multiple Groups Multiply Read Traffic



At 120 MB/s ingress and four full-volume groups: $120 \times 4 = 480$ MB/s of logical consumer reads — before protocol and replication overhead. Partition count must fit broker network and disk-read capacity, not only producer throughput.

Record Rate vs Byte Rate: Both Matter

Workload A — High Record Rate

$100,000 \text{ rec/s} \times 1 \text{ KB} = 100 \text{ MB/s}$

- High request and serialisation overhead
- Per-record metadata pressure
- Batching is critical

Workload B — Large Records

$1,000 \text{ rec/s} \times 100 \text{ KB} = 100 \text{ MB/s}$

- Memory and network burst pressure
- Fetch-size configuration is critical
- Compression effectiveness differs

Both produce identical aggregate MB/s but stress Kafka very differently. Benchmark with realistic keys, headers, sizes and batch behaviour.

Compression Changes the Numbers

Raw traffic

200 MB/s

Compression ratio

2.5:1

Stored / transmitted

$$\frac{200}{2.5} = 80 \text{ MB/s}$$

But compression effectiveness varies with record similarity, batch size, codec choice, producer linger settings and whether payloads are already compressed. Measure actual producer and broker byte metrics during load tests — do not assume the ideal ratio.

Replication Multiplies Internal Cluster Traffic

For orders.events at RF 3

$120 \text{ MB/s} \times 3 \approx$
360 MB/s total internal traffic

Each leader write must reach two followers.

This is a planning approximation — account for:

- Protocol overhead
- Acknowledgements
- Compression and batching
- Cross-rack or cross-AZ traffic
- Consumer read traffic on top

Storage Sizing: Where Does 12 TB Come From?

Approximate retained primary data:

$$\text{Primary} = \text{ingress} \times \text{retention seconds}$$

$$= 20 \text{ MB/s} \times 604,800 \text{ s} \approx 12.1 \text{ TB}$$

With replication factor 3:

$$\text{Replicated} = 12.1 \times 3 \approx 36.3 \text{ TB}$$

Partitions	Avg size per partition
6	~2 TB
12	~1 TB
24	~500 GB

Very large partitions take longer to copy during reassignment and recovery. Very small partitions create metadata, file and operational overhead.

Recovery Time Is a Hard Constraint

$$\text{Recovery time} = \frac{\text{Partition replica size}}{\text{Safe transfer rate}}$$

Partition replica

500 GB

Safe transfer rate

50 MB/s

Estimated time

≈ 2.8 hours

Real recovery is often slower — production traffic, disk contention and multiple replicas share the same resources simultaneously. Size partitions so that recovery completes within your RTO before taking on new traffic.

Growth Headroom: Build It In Early

Current calculation → 12 partitions

Expected annual growth → 40 %

$$12 \times 1.4 = 16.8 \rightarrow \text{choose 18 partitions}$$

Validate broker impact before committing.

What headroom must cover

- Forecast traffic growth
- Seasonal and campaign peaks
- Product launches
- New consumer applications
- Operational uncertainty

 Avoid extreme over-partitioning "just in case." More partitions are not free — see the costs on a later card.

The Hot Partition Problem

A topic can look healthy overall while one partition is overloaded.

→ Causes

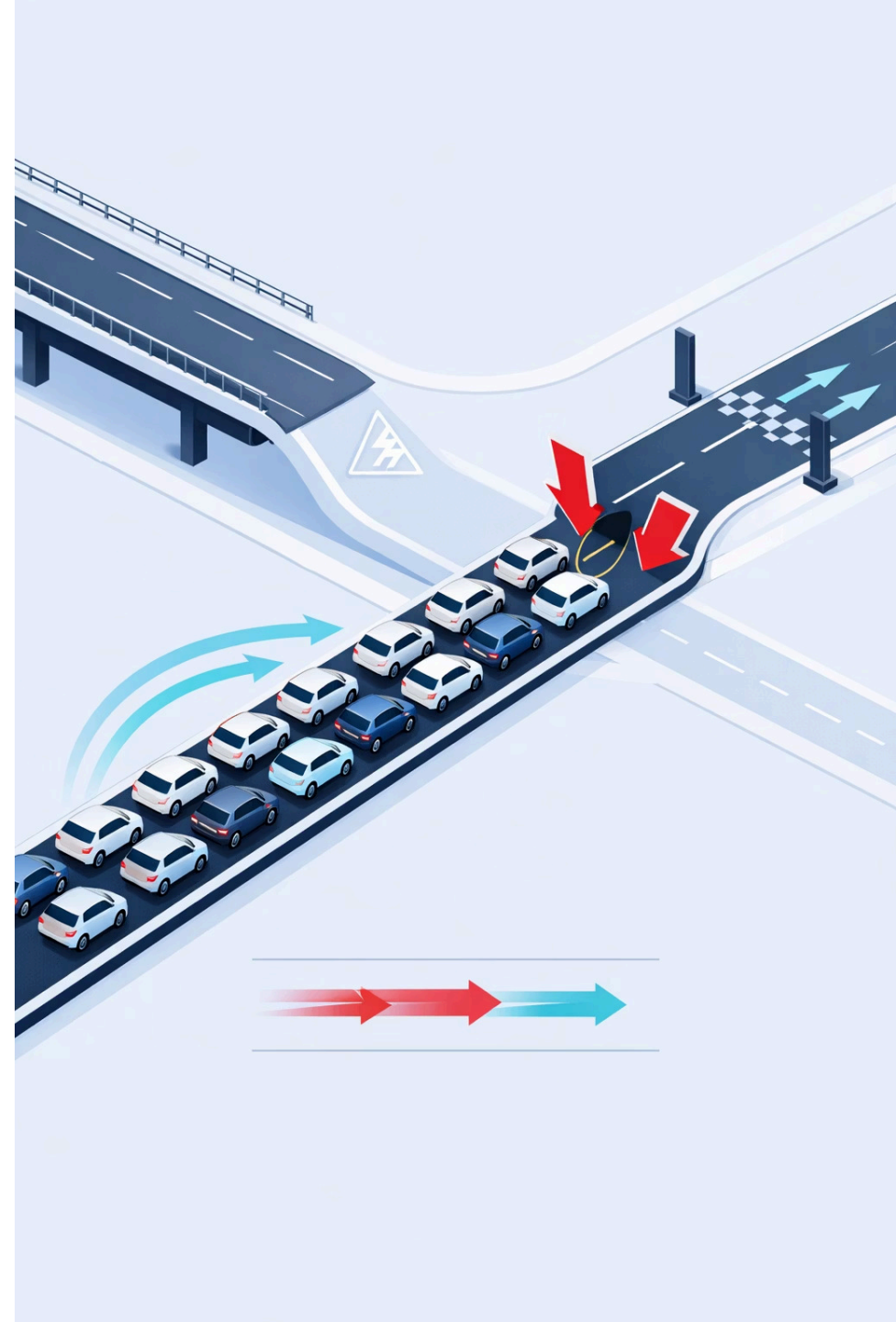
Low-cardinality keys · One very active tenant · Time-based keys concentrating current traffic · Custom partitioner defect

→ Symptoms

Uneven bytes-in across brokers · Higher latency for one partition · Consumer lag on one partition · One broker hits disk or network limits first

→ Fix

Adding partitions alone does not fix poor key distribution — fix the key strategy first



Key Cardinality Sets a Practical Ceiling

The problem

24 partitions but only 5 country keys produce nearly all traffic → only 5 partitions ever receive meaningful load

Better key strategies

- Higher-cardinality business key: `orderId`
- Composite: `country` + `customerId`
- Intentional bucket: `tenantId` + `bucket`

Never change key strategy without reviewing ordering requirements.

Skew Safety Factor: Size for the Hottest Partition

Average planned load

10 MB/s per partition

Observed skew factor

1.8×

Hottest partition load

~18 MB/s

Tested safe capacity

15 MB/s

- ❌ The design is unsafe even if total topic throughput looks acceptable. Always measure per-partition rates during realistic peak traffic — not just aggregate topic rates.

More Partitions Are Not Free

- Controller metadata load
Every partition is a metadata entry; very large counts stress the controller
- Open files and log segments
Each partition has at least one active segment file and index
- Leader-election work
More partitions = more elections after broker failure
- Recovery time
More replicas to copy when a broker returns

Avoid any rule like "always create 50 partitions." Partition count must follow workload evidence and cluster capacity.

Partition Planning Decision Worksheet

Decision Field	Value for orders.events
Peak records/s	60,000
Peak compressed MB/s	120 MB/s
Tested safe write MB/s/partition	15 MB/s
Write-side P requirement	8
Slowest consumer MB/s/instance	10 MB/s
Read-side P requirement	12
Skew factor	1.5×
Growth headroom (40 %)	$12 \times 1.4 \rightarrow 18$
Proposed partitions	18
Avg retained partition size	~670 GB
Estimated replica recovery time	~3.7 hrs @ 50 MB/s

Store this worksheet with the topic creation request as part of your infrastructure change record.

Leader · Follower · ISR

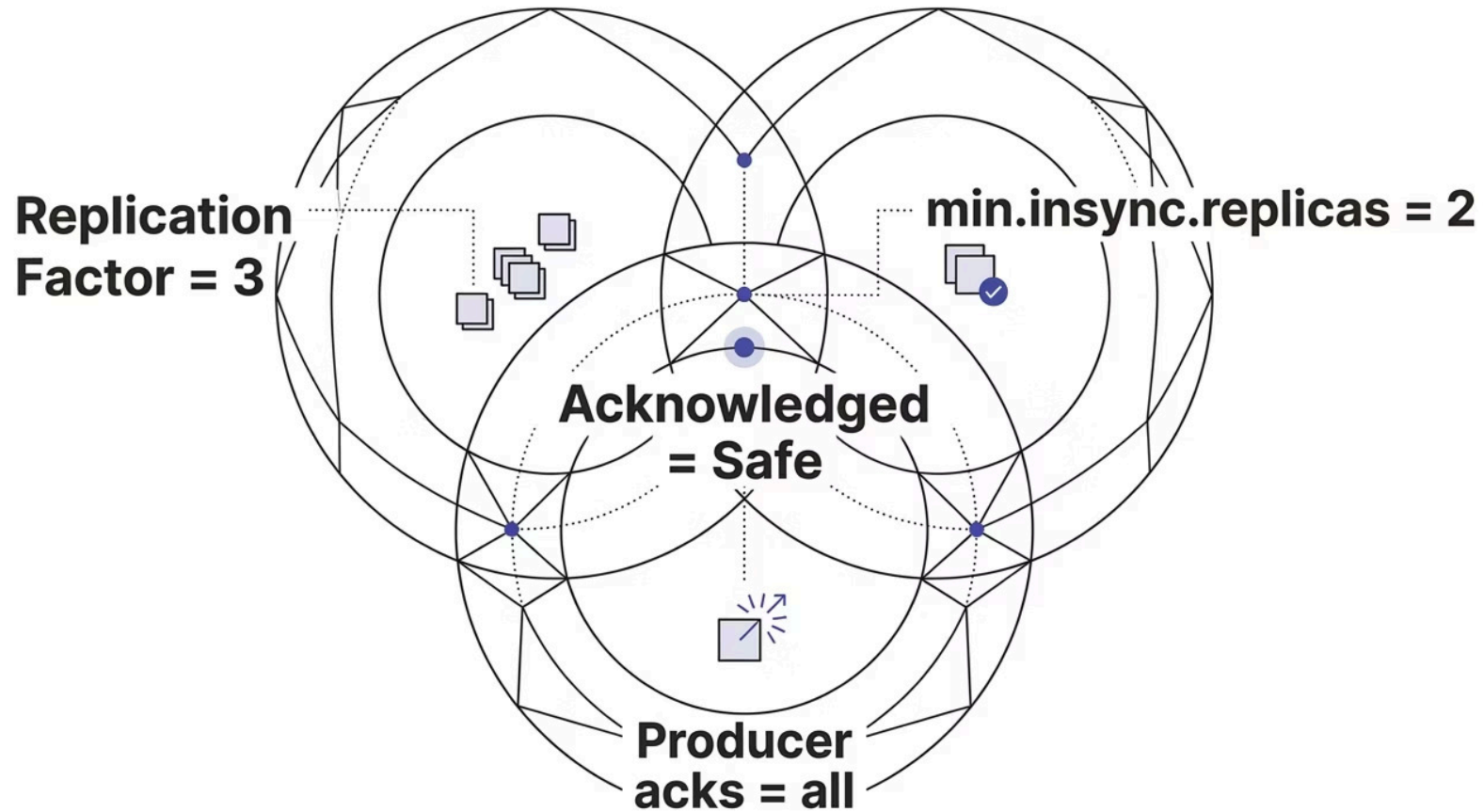
Replication is how Kafka survives broker failures without losing acknowledged data.

Leader, Follower and ISR Explained



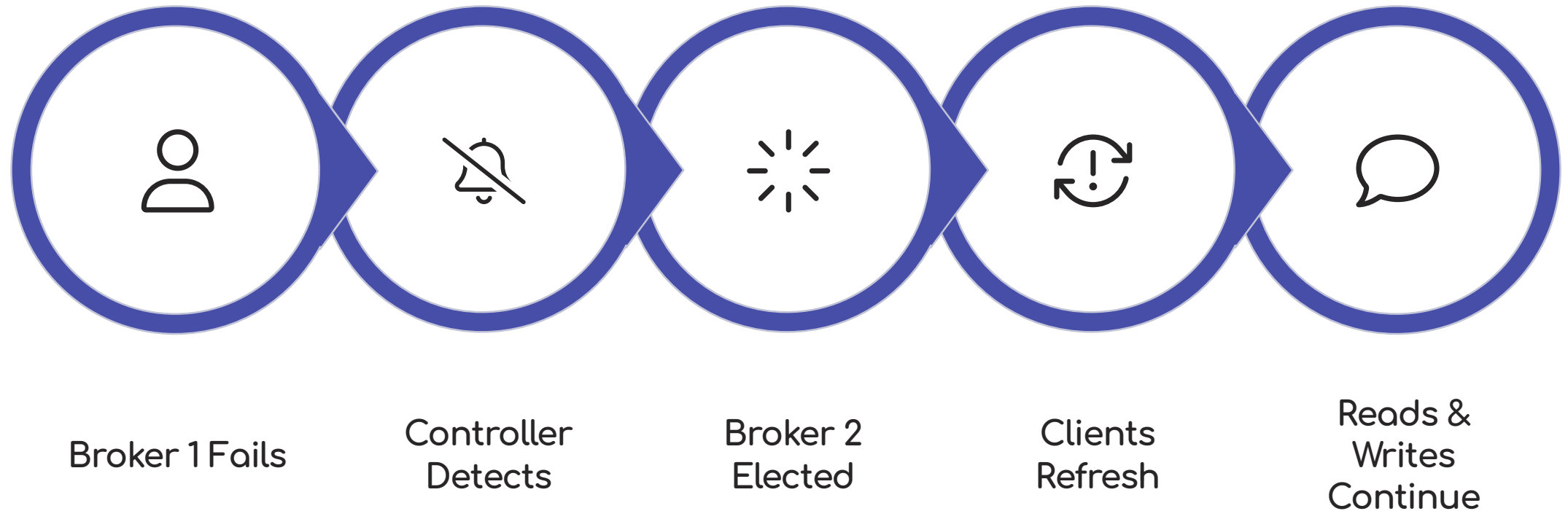
ISR (In-Sync Replicas) is the set of replicas sufficiently caught up with the leader. Healthy: Replicas: 2,3,1 ISR: 2,3,1. Degraded: Replicas: 2,3,1 ISR: 2,3 — one replica is lagging or gone.

The Durability Triangle



Effect: A write is acknowledged only when the ISR condition is fully satisfied. If ISR falls below 2, new writes fail rather than silently reducing durability. Availability is intentionally traded for protection against data loss on acknowledged records.

What Happens When a Broker Fails?



Recovery quality depends on replication health before the failure. If a follower was lagging before the crash, it may not be eligible for election, extending downtime or forcing a trade-off between availability and data safety.

ISR Shrink: An Early Warning, Not a Routine Event



Broker failure or restart

The most obvious cause — replica goes offline completely



Network latency or packet loss

Follower cannot keep up with leader; lag exceeds the `replica.lag.time.max.ms` threshold



Slow or failed disk

I/O saturation on the follower broker delays log writes



Long GC pauses

Stop-the-world pauses cause the replica to fall behind temporarily

 Do not treat ISR shrink as harmless. Under-replication is an operational warning that deserves immediate investigation.

Replicas vs ISR vs Offline: Do Not Confuse Them

Field	Meaning
Replicas	All assigned broker copies for this partition
ISR	Subset currently considered in sync with the leader
Leader	The single replica accepting reads and writes
Offline Replicas	Assigned replicas currently unreachable

orders.events partition 0: Leader: 2 Replicas: 2,3,1 ISR: 2,3 → Broker 1 replica is out of sync. One degree of redundancy is lost. Writes with `acks=all` still succeed ($\text{ISR} \geq \text{min.insync.replicas}=2$) — but one more failure could halt writes.

Unclean Leader Election: Availability vs Durability

`unclean.leader.election.enable = false`
(recommended)

Partition stays unavailable until an in-sync replica is eligible. Durability is preserved. No acknowledged data is lost.

`unclean.leader.election.enable = true`

An out-of-sync replica is elected. Partition becomes available. Acknowledged records that the new leader never received are permanently lost.

- ⊗ Do not enable unclean election casually at topic or broker level. Define the policy per workload criticality and document recovery expectations with application owners.

Rack-Aware Placement: Survive an AZ Failure



Set a distinct `broker.rack` for each failure domain (availability zone, data-centre rack, cloud failure domain). With $RF=3$ across three racks, Kafka attempts to spread replicas across racks automatically. Validate that rack labels reflect real infrastructure topology — a mislabelled rack gives false confidence.

Checkpoint: The ISR Scenario

Configuration: `RF=3`, `min.insync.replicas=2`, `acks=all`

Situation: Two brokers fail. Only one replica remains. That replica is the partition leader.

Question: Can producers still write new records to this partition?

A — Yes

Leader is alive, so writes succeed

B — No

ISR size (1) is below `min.insync.replicas` (2), so writes with `acks=all` are rejected

C — Depends

Only if the producer switches to `acks=1`

Discuss. Answer on the next card.

Answer: Writes Fail — Intentionally

Correct answer: B

The leader is alive, so reads of committed data may continue. But `acks=all` writes require $ISR \geq \text{min.insync.replicas}$. $ISR=1 < 2 \rightarrow$ new writes are rejected with `NOT_ENOUGH_REPLICAS`.

This is a safety mechanism, not a bug

The correct response is to restore replica health — not weaken durability settings under pressure. Restore the failed brokers and let replicas catch up.

Retention · Compaction · Segments

Configuration controls how data lives, ages and is cleaned — affecting storage, cost and consumer behaviour.

Retention: Delete Policy

Time-Based vs Size-Based Retention



Time-based: retention.ms

Seven days:
`retention.ms=604800000`

- Enforced at log-segment level
- Records not deleted at individual expiry — segment must become inactive first
- Longer retention → more disk → longer recovery

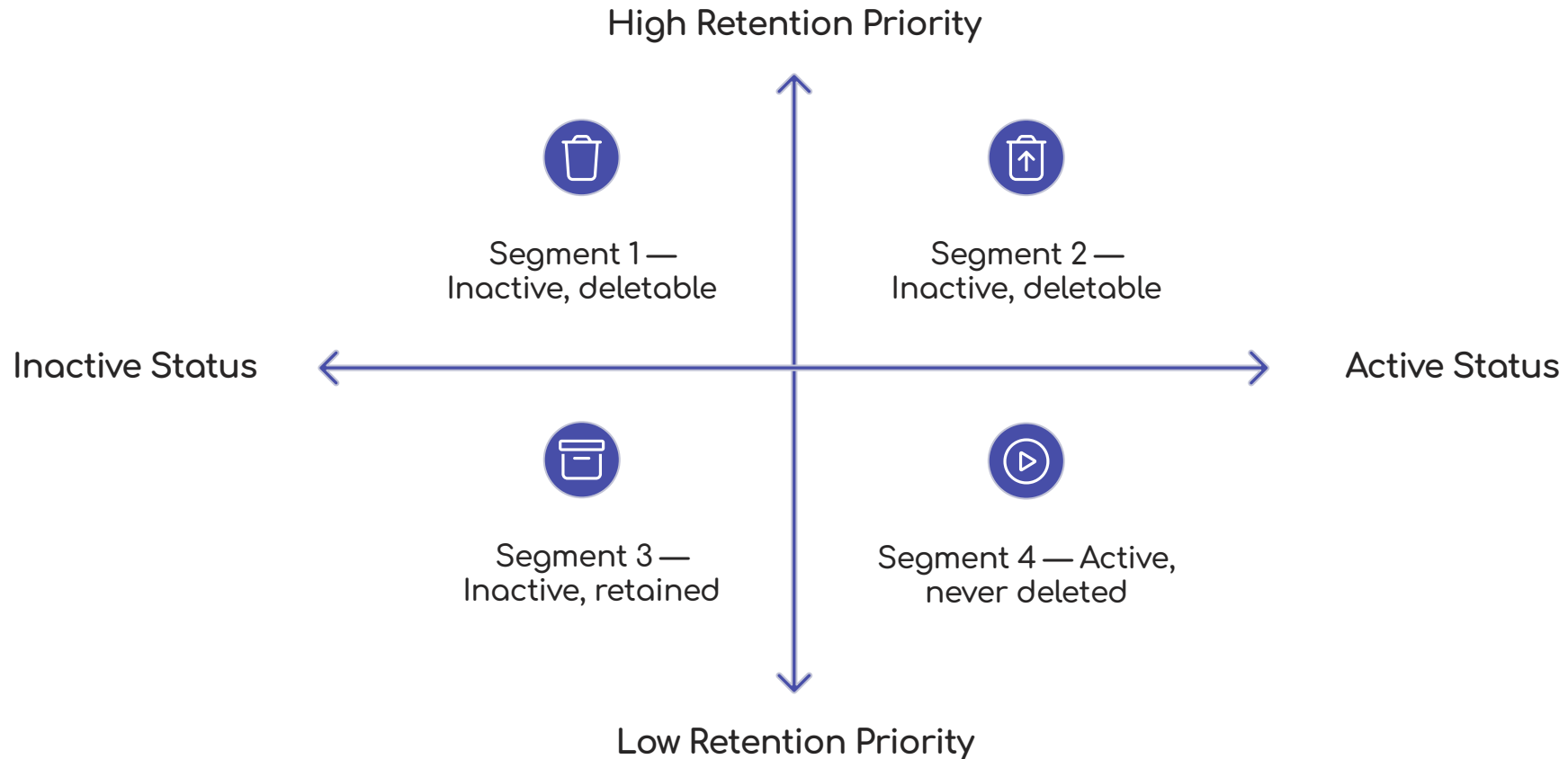
Size-based: retention.bytes

Limit per partition, not the whole topic

Topic limit $\approx$ retention.bytes $\times$ partitions $\times$ RF

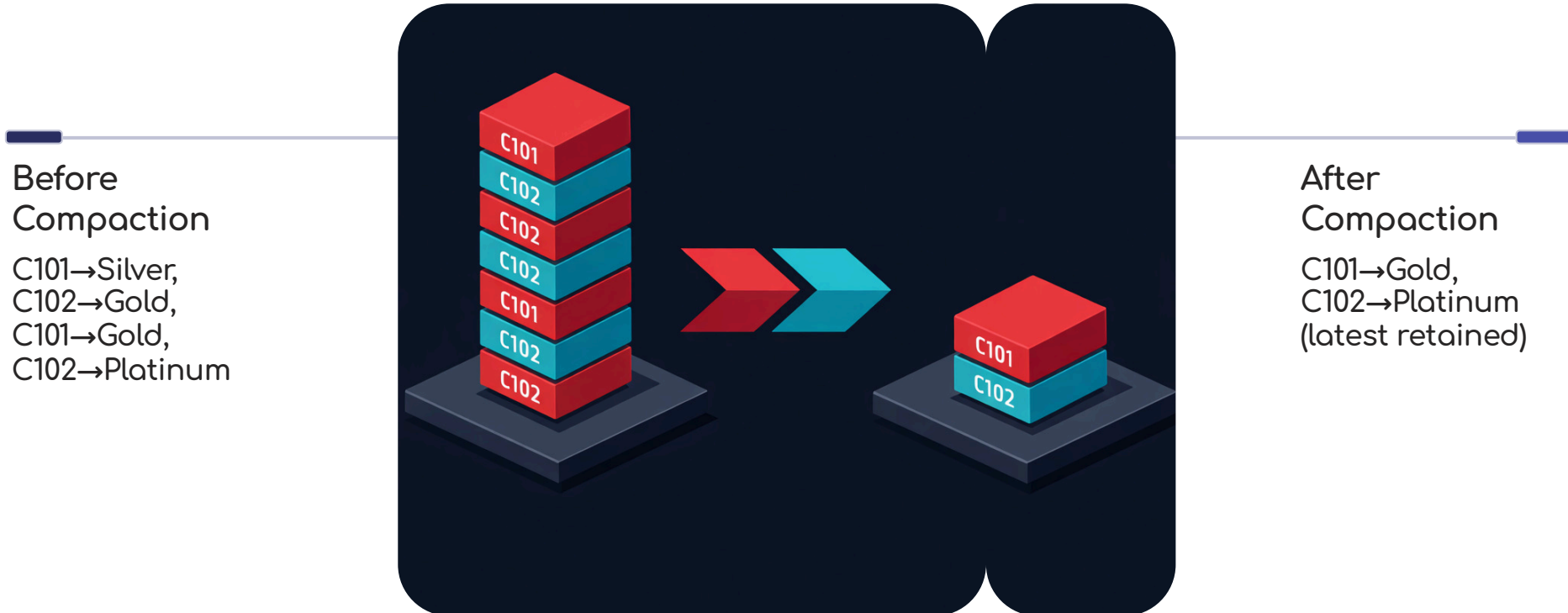
- High-volume bursts shorten the effective time window
- Use carefully when time-based semantics are expected

Log Segments: Retention's Unit of Work



Retention deletes whole inactive segments. The active segment is never deleted. Smaller segments improve cleanup responsiveness. Too many small segments increase file handles, index files and metadata overhead. Key settings: `segment.bytes`, `segment.ms`, `segment.jitter.ms`.

Log Compaction: Retain the Latest Value Per Key



Compaction does not mean "one record immediately." Duplicates can remain until the log cleaner runs. Record order and offsets are preserved for retained records. Keys are essential — records without keys are not eligible for deduplication. Use for state/changelog topics, not event history.

Tombstones: Deleting from Compacted Topics

What is a tombstone?

A record with a key and null value: C101 → null

Meaning: remove this key from the compacted state entirely.

`delete.retention.ms` controls how long tombstones remain before the cleaner may remove them.

Operational risks

- Consumers offline longer than the tombstone window may rebuild incorrect state
- Missing keys prevent meaningful compaction
- Application teams must distinguish a null value from an absent event

Combining Policies: compact,delete

`cleanup.policy=delete`

Event history topics. Seven-day replay window. Segments age out by time or size.
`orders.events` uses this.

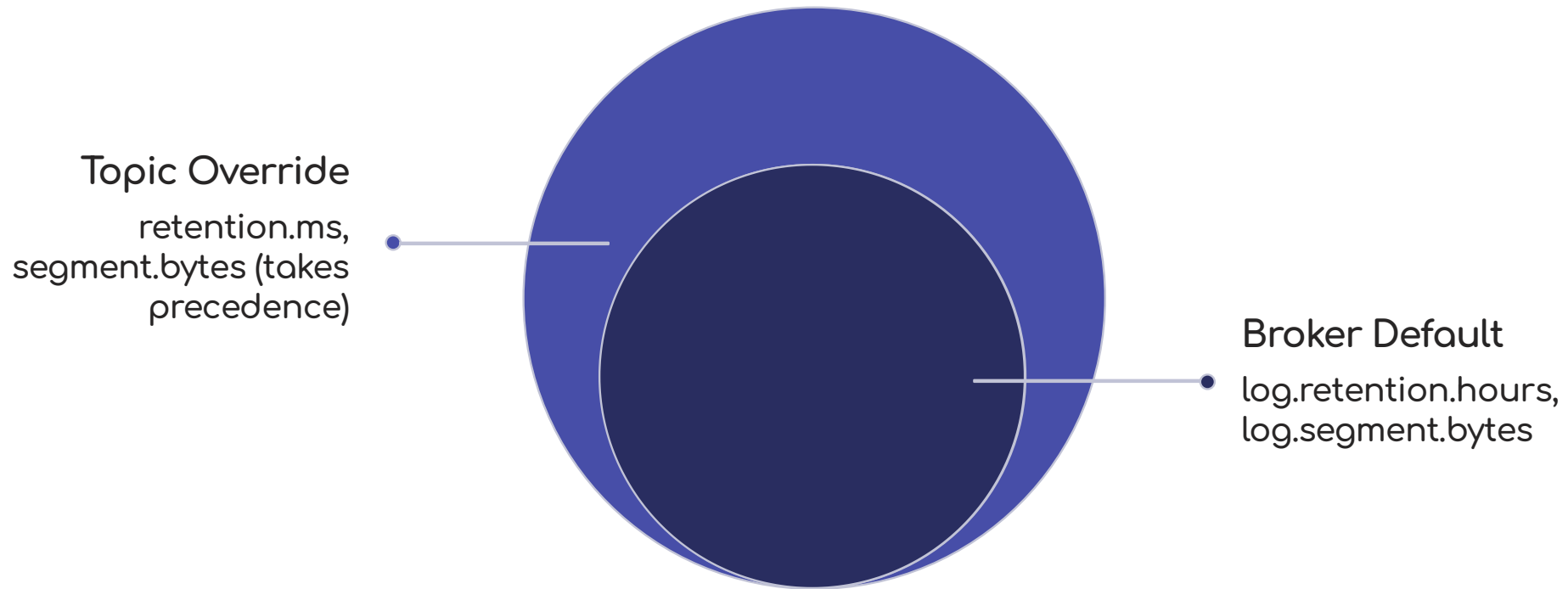
`cleanup.policy=compact`

State/changelog topics. Latest value per key forever.
`customer.state.lab` uses this.

`cleanup.policy=compact,delete`

Bounded changelog: latest state retained, but data older than the retention window is eventually removed. Best of both worlds for many use cases.

Topic Configuration Precedence



Do not assume every topic uses the broker default. Inspect dynamic overrides explicitly. Administrators must also check whether old emergency overrides have become permanent — a common source of configuration drift across topics with inconsistent policies.

Create · Inspect · Modify

Production topic operations require explicit settings, version control and rollback planning.

Creating a Topic: Use Explicit Production Settings

```
bin/kafka-topics.sh \  
  --bootstrap-server kafka-1:9092 \  
  --create \  
  --topic orders.events \  
  --partitions 12 \  
  --replication-factor 3 \  
  --config min.insync.replicas=2 \  
  --config cleanup.policy=delete \  
  --config retention.ms=604800000
```

- Capture in version control
Store the exact command in automation — not in a terminal history
- Verify after creation
Immediately describe the topic and confirm assignment, replication and effective config
- Treat as infrastructure change
Require peer approval, an approved window and a rollback path

Describing and Inspecting Topics

```
bin/kafka-topics.sh --bootstrap-server kafka-1:9092 \  
--describe --topic orders.events
```

Topic: orders.events Partition: 0 Leader: 2
Replicas: 2,3,1 Isr: 2,3,1

Leader: -1

No active leader → partition
unavailable

ISR < Replicas

Under-replicated → reduced
redundancy

Same broker leads many

Leadership skew → uneven CPU
and network load

Filtering for Problem Partitions

Under-replicated (ISR smaller than replicas)

```
bin/kafka-topics.sh --bootstrap-server kafka-1:9092 \  
--describe --under-replicated-partitions
```

No active leader

```
bin/kafka-topics.sh --bootstrap-server kafka-1:9092 \  
--describe --unavailable-partitions
```

ISR below configured minimum

```
bin/kafka-topics.sh --bootstrap-server kafka-1:9092 \  
--describe --under-min-isr-partitions
```

- ① Use monitoring for continuous detection. CLI checks are valuable for targeted investigation — not as a substitute for automated alerting.

Modifying Topic Configuration

```
# Add or change settings
bin/kafka-configs.sh \
  --bootstrap-server kafka-1:9092 \
  --entity-type topics \
  --entity-name orders.events \
  --alter \
  --add-config retention.ms=1209600000,min.insync.replicas=2
```

Remove override — return to broker default

```
bin/kafka-configs.sh \
  --bootstrap-server kafka-1:9092 \
  --entity-type topics \
  --entity-name orders.events \
  --alter \
  --delete-config retention.ms
```

 Always record the previous value and the rollback command before executing any configuration change in production.

Increasing Partition Count: Do It Once, Do It Right

```
bin/kafka-topics.sh \  
  --bootstrap-server kafka-1:9092 \  
  --alter \  
  --topic orders.events \  
  --partitions 18
```

Supported

Increase partition
count

Not supported

Decrease partition
count

Not automatic

Existing data is not
redistributed

Application-visible

Future keyed records
may map differently;
consumer groups
rebalance

Replica Reassignment: Six Operational Phases



Capture

Save the current assignment as your rollback baseline



Plan

Build and peer-review the proposed assignment JSON



Execute with throttle

Apply with a safe `--throttle` to protect production traffic



Monitor

Watch replication lag, latency, under-replicated partitions and broker utilisation



Verify

Confirm every partition reached its intended assignment



Clean up throttles

Remove throttle configs immediately after completion

Reassignment Commands

```
# Execute with 50 MB/s throttle  
bin/kafka-reassign-partitions.sh \  
  --bootstrap-server kafka-1:9092 \  
  --reassignment-json-file reassignment.json \  
  --execute \  
  --throttle 50000000
```

```
# Verify progress  
bin/kafka-reassign-partitions.sh \  
  --bootstrap-server kafka-1:9092 \  
  --reassignment-json-file reassignment.json \  
  --verify
```

Throttle too high

Client latency spikes, broker saturation, ISR instability

Throttle too low

Reassignment runs for hours or days, extended vulnerability

Topic Deletion: Safety First

```
bin/kafka-topics.sh \  
  --bootstrap-server kafka-1:9092 \  
  --delete \  
  --topic orders.events
```

Before you delete

Confirm exact cluster name and topic name · Verify active consumer groups · Check retention/compliance obligations · Export required data

Governance controls

Require owner approval · Use least-privilege ACLs for delete permission · Prefer automation with review gates

Never

Use broad shell patterns in destructive topic workflows. One typo deletes the wrong topic permanently.

Naming · ACLs · Lifecycle

Governance prevents the operational chaos that accumulates over years of uncontrolled topic growth.

Topic Naming Standards

Recommended pattern

`<domain>.<entity>.<type>.<version>`

- `commerce.orders.events.v1`
- `payments.transactions.events.v1`
- `inventory.stock.compacted.v1`

Standards must define

- Allowed characters and case
- Environment separation (prod vs dev)
- Ownership and data classification metadata
- Dead-letter and retry topic suffixes
- Internal and reserved prefixes

❌ Example risk: application writes to `order.events` instead of `orders.events`, silently creating a topic with broker defaults and no owner.

Preventing Uncontrolled Topic Creation

01

Disable auto.create.topics.enable

Prevents typos from silently spawning unintended topics in production

02

Create through reviewed automation

All topics created via pipeline with explicit partition, replication and retention settings

03

Require metadata

Ownership, data classification and cost centre attached at creation time

04

Detect drift continuously

Automated scan for topics that do not match naming standards or have no registered owner

Topic-Level Security

Separate identities

Application service accounts must never share administrator credentials

Least privilege

Grant Read/Write per topic. Restrict Alter, Delete and cluster-level operations to named administrator accounts only

Use naming patterns carefully

Wildcard ACLs are convenient but can inadvertently grant access to future topics

Audit all admin changes

Log every Create, Alter, Delete and ACL change. Review regularly for unexpected patterns

Detect · Diagnose · Resolve

Healthy topics are the foundation of a healthy Kafka platform.

Core Topic Health Signals to Monitor

Availability signals

- Offline partition count → must be 0
- Under-replicated partition count → must be 0
- Under-min-ISR partition count → must be 0
- ISR shrink/expand rate → should be near 0

Performance signals

- Bytes in/out by topic and broker
- Produce and fetch request latency
- Consumer lag by group/topic/partition
- Log size and disk utilisation
- Leader count balance across brokers

❏ Alerts should indicate both technical severity and the affected business topics — "under-replicated: orders.events" is more actionable than a generic broker metric.

Partition State: Healthy, Degraded, Unavailable

State	Example	Operational Meaning
Healthy	Replicas 1,2,3; ISR 1,2,3	Full redundancy — all good
Degraded	Replicas 1,2,3; ISR 1,2	Service works, redundancy reduced — investigate now
Under min-ISR	ISR < min.insync.replicas	acks=all writes may fail — urgent
Unavailable	Leader: -1	Partition cannot serve — critical

Do not wait for unavailability. Under-replication is your early warning to act before a second failure turns degraded into unavailable.

Troubleshooting Under-Replicated Partitions

1 Identify scope

Which topics, partitions and brokers are affected? Is this one partition or many?

2 Check broker health

Is a broker offline, restarting or overloaded? Check disk capacity, I/O errors, network and JVM pauses

3 Inspect replication lag

Is the follower fetching at all? Is it catching up or still diverging?

4 Check recent changes

Recent reassignments, configuration changes or broker restarts — correlation is often causal

5 Verify ISR recovery

Confirm replica rejoins ISR after the root cause is resolved. Do not close the incident until ISR is fully restored.

Troubleshooting: Disk Growth and Partition Skew

Disk growth investigation

- Which topics and partitions consume the most space?
- Was `retention.ms` recently increased?
- Is `retention.bytes` unlimited?
- Are segments rolling? Is the log cleaner falling behind?
- Did a reassignment temporarily duplicate data?



Do not delete Kafka log files manually while the broker is running.

Partition skew remedies

- Record-count skew → fix producer key strategy
- Leader skew → preferred leader election
- Replica-count skew → replica reassignment
- Traffic skew → fix keys before adding partitions

Diagnose the type of skew before choosing a remedy.

Lab Time — Practise Safe Operations

Three labs. Use your non-production cluster. Document every command you run.

Lab 1 — Topic Lifecycle

01

Create

Create `orders.events` with 6 partitions, RF 3, 7-day retention, `min.insync.replicas=2`

02

Inspect

List and describe the topic. Verify leaders, assignment and effective config

03

Produce

Send keyed order events. Observe which key lands on which partition and at what offset

04

Expand

Increase partitions from 6 to 9. Observe future key placement change and consumer rebalance

05

Modify and clean up

Change retention to 14 days. Then delete the override and verify inherited broker default. Delete a disposable test topic safely.

Lab 2 — Failure and Recovery

01

Baseline

Identify leaders and ISR for every partition of `orders.events`

02

Induce failure

Stop a broker hosting multiple partition leaders

03

Observe

Watch leader movement, ISR shrink and producer behaviour with `acks=all`

04

Compare

Check current ISR against `min.insync.replicas`. Do writes succeed or fail? Why?

05

Recover

Restart the broker. Observe replica catch-up, ISR expansion and whether leadership rebalances automatically

❏ Expected insight: Replication health *before* a failure determines recovery quality and speed.

Lab 3 — Retention and Compaction

Create two topics

- `orders.history.lab` — `cleanup.policy=delete`
- `customer.state.lab` — `cleanup.policy=compact`

Exercises

- Produce repeated keys with updated values
- Produce tombstones to the compacted topic
- Inspect partition log directories in the lab environment
- Observe segment rolling and asynchronous cleanup

Document your answers

- Why is cleanup not immediate?
- What happens to a consumer that reads after a tombstone is cleaned?
- When would you choose compact over delete?
- When would you combine both policies?

Topic Design Review Checklist

Identity and ownership

Clear name, business owner,
technical owner, data
classification and access policy

Throughput and sizing

Partition count with calculation,
replication factor, rack
placement and broker capacity
check

Durability

min.insync.replicas, producer
acks and unclean election
policy documented

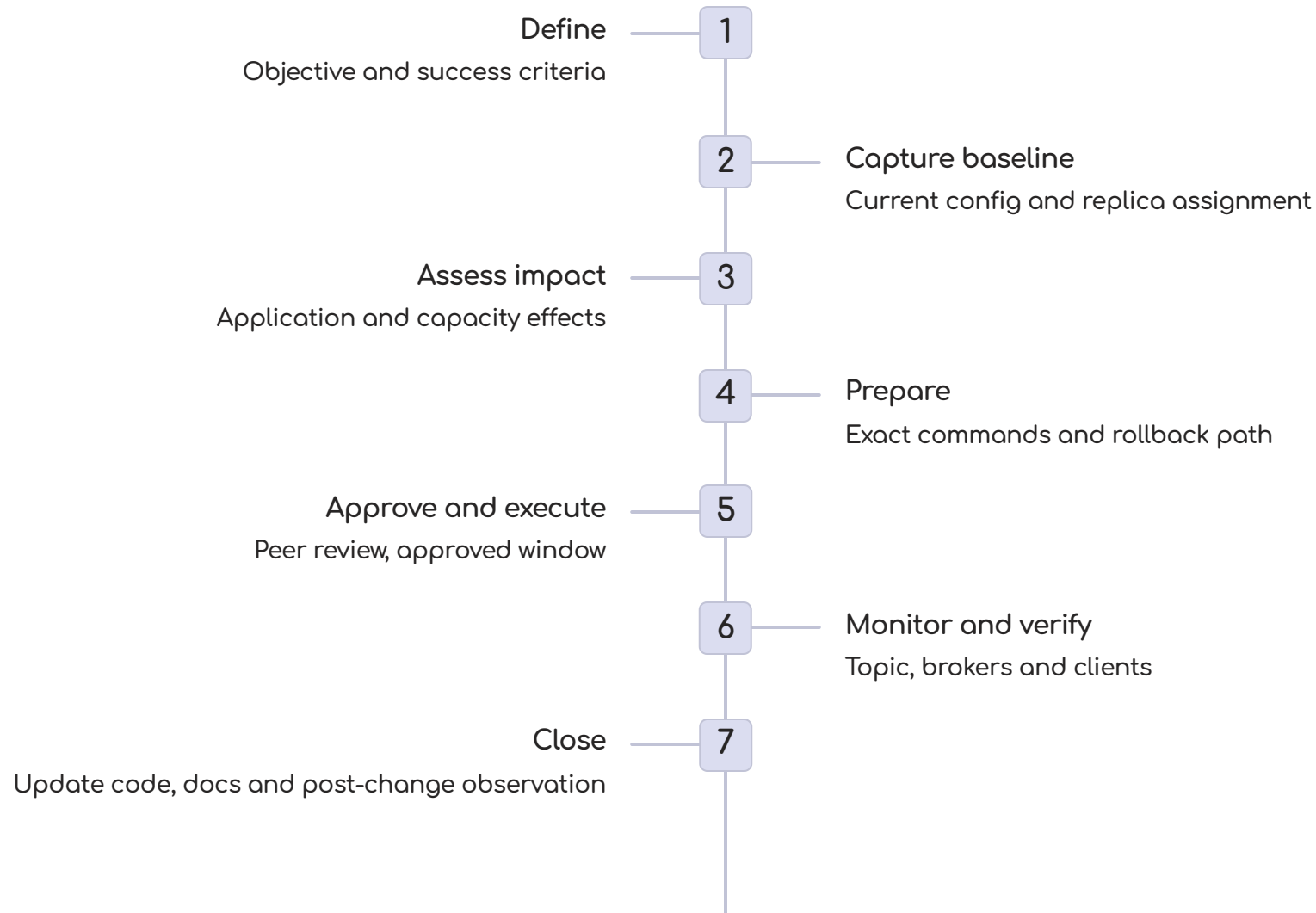
Data lifecycle

Cleanup policy, retention period, compaction and
tombstone semantics confirmed

Operations and cost

Monitoring, alerting, storage cost estimate and
deletion/lifecycle plan in place

Production Change Runbook



Ten Key Takeaways

01

Topic = distributed partition logs

Not a queue — data retention is independent of consumer progress

03

Partition count is a capacity decision

Use write/read formulas, benchmarks, skew factors and growth headroom

05

RF + min.insync.replicas + acks work together

All three must be set deliberately; none guarantees durability alone

07

Retention and compaction solve different problems

Delete for event history; compact for latest state; combine for bounded changelogs

09

Replication factor changes require reassignment

Use throttling, monitor progress and clean up throttle configs after completion

02

Ordering is per-partition only

Use orderId as key to guarantee order within a partition

04

More partitions are not free

Controller load, files, recovery time and complexity all increase

06

ISR shrink is an early warning

Investigate immediately — the next failure could halt writes or create data loss

08

Partition count can increase, never decrease

Plan adequate headroom at creation — keyed order semantics are affected by expansion

10

Topic operations must be automated and governed

Healthy topics are the foundation of every healthy Kafka platform

Thank You — Questions and Discussion

Official References

Apache Kafka Documentation: kafka.apache.org/documentation ·

Topic Configs: kafka.apache.org/configuration/topic-configs ·

Kafka 4.x Operations: kafka.apache.org/43/operations

Next Recommended Module

Kafka Broker Configuration, Storage and Cluster Operations —
building on everything covered in this session

- ❏ Use the documentation version that matches the Kafka release deployed in your environment.

